

Санкт-Петербургский государственный университет
Кафедра системного программирования

Подсчёт треугольников в графе: GraphBLAS vs SPLA

Немакин Никита Антонович, группа 23.Б11-мм

27 сентября 2026 г.

Санкт-Петербург
2026

Постановка задачи

Дан неориентированный граф $G = (V, E)$ без петель и кратных рёбер.

Задача: Найти количество треугольников в графе, то есть таких троек вершин (u, v, w) , что между каждой парой вершин существует ребро: $(u, v) \in E$, $(v, w) \in E$, $(w, u) \in E$.

Порядок вершин в тройке не важен.

Применения:

- Анализ социальных сетей
- Оценка кластеризации графа
- Поиск сообществ

Пусть A — симметричная матрица смежности графа. Тогда

$$T = \frac{1}{6} \sum_{i,j} (A^2 \odot A)_{ij}$$

Здесь $\odot$ — поэлементное умножение, а A используется как маска.

- $(A^2)_{ij}$ равно числу общих соседей вершин i и j
- Маска оставляет только пары (i, j) , соединённые ребром
- Каждый треугольник учитывается дважды для каждой ориентированной пары двух его вершин, поэтому сумма делится на 6

Пусть U — строго верхняя треугольная часть матрицы смежности. Тогда

$$T = \sum_{i,j} (U^2 \odot U)_{ij}$$

Здесь $\odot$ — поэлементное умножение, а U используется как маска.

- Матрица U задаёт единственную ориентацию каждого ребра: от меньшего индекса к большему
- Для треугольника $i < j < k$ произведение находит путь $i \rightarrow j \rightarrow k$, а маска проверяет ребро $i \rightarrow k$
- Каждый треугольник учитывается ровно один раз, поэтому деление не требуется

Используемые библиотеки:

- SuiteSparse:GraphBLAS — полная реализация стандарта GraphBLAS, используемая для работы с графовыми алгоритмами на языке линейной алгебры
- SPLA — открытая библиотека для работы с разреженными матрицами, предоставляющая поддержку GPU

Реализация использует маскированное умножение разреженных матриц в арифметическом полукольце $(\mathbb{N}, +, \times, 0, 1)$ с последующей редукцией (суммированием).

- **Цель:** Сравнение реализаций алгоритмов Burkhardt и Sandia на SuiteSparse:GraphBLAS и SPLA (GPU/CPU)
- **Шаги:**
 - ▶ Провести 20 запусков алгоритмов
 - ▶ Вычислить средние значения и построить доверительные интервалы
 - ▶ Проанализировать результаты
- **Вопросы:**
 - ▶ Время выполнения — алгоритм $\times$ бэкенд + общее сравнение
 - ▶ Потребление памяти — peak RSS + peak heap size

Эксперимент: датасет

В качестве тестовых данных для эксперимента были выбраны графы из набора SNAP¹.

Название	Вершин	Рёбер
loc-Brightkite	58,228	214,078
com-Amazon	334,863	925,872
amazon0312	400,727	3,200,440
roadNet-CA	1,965,206	2,766,607
com-LiveJournal	3,997,962	34,681,189

Основания для выбора:

- Популярность набора SNAP
- Разнообразие предметных областей
- Графы различных размеров и плотности

¹Stanford Large Network Dataset Collection: <https://snap.stanford.edu/data/>

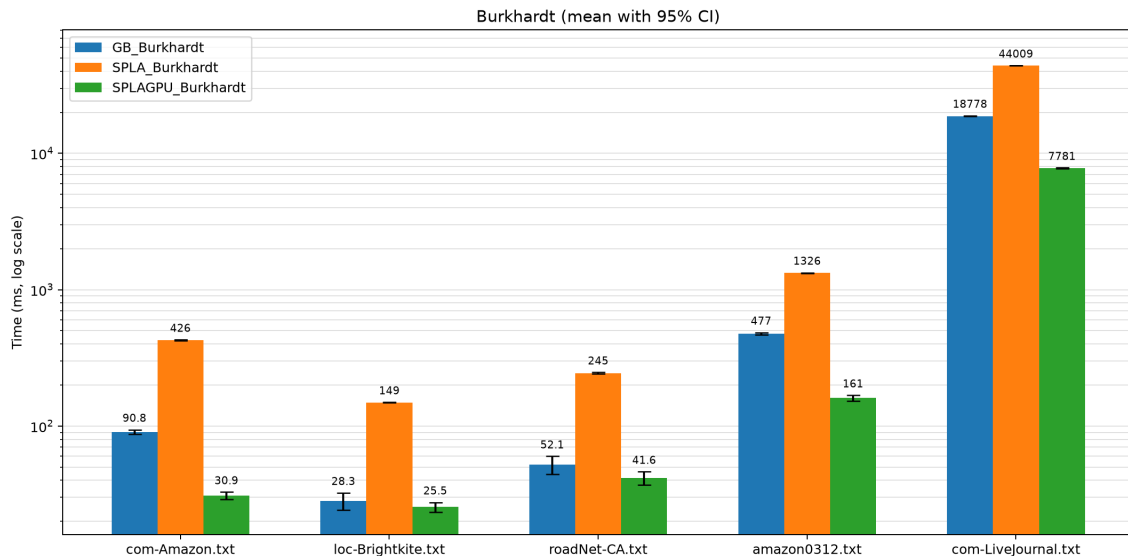
Характеристики машины:

- CPU: Intel i5-1135G7, 4 cores, 8 threads @ 4.2 GHz
- GPU: Intel TigerLake-LP GT2
- RAM: 16 GB DDR4 @ 3200 MHz

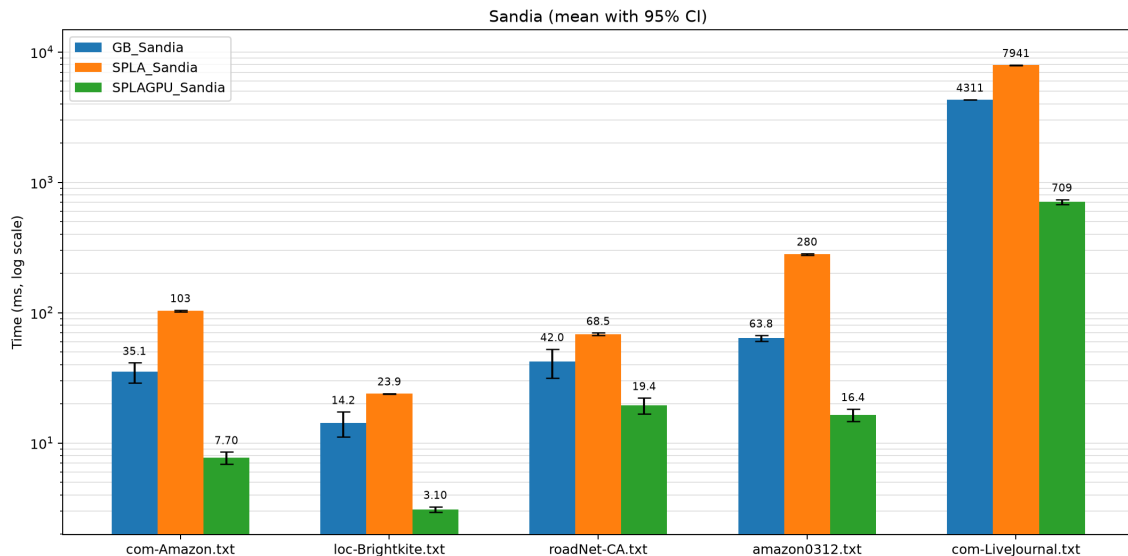
ПО:

- OS: Ubuntu 24.04.5 LTS
- gcc: v13.3.0
- OpenCL: intel-ocl-runtime v23.43.27642

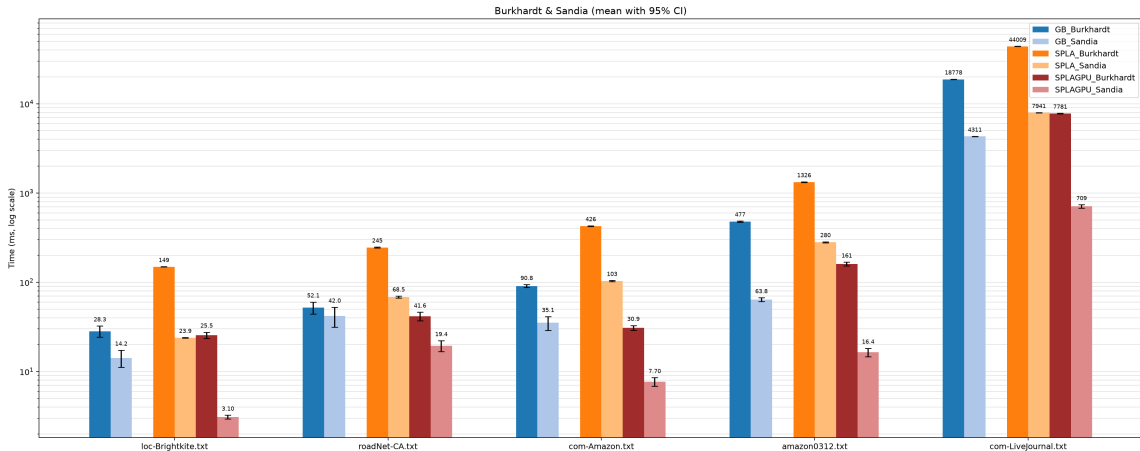
Эксперимент: Burkhardt — время выполнения



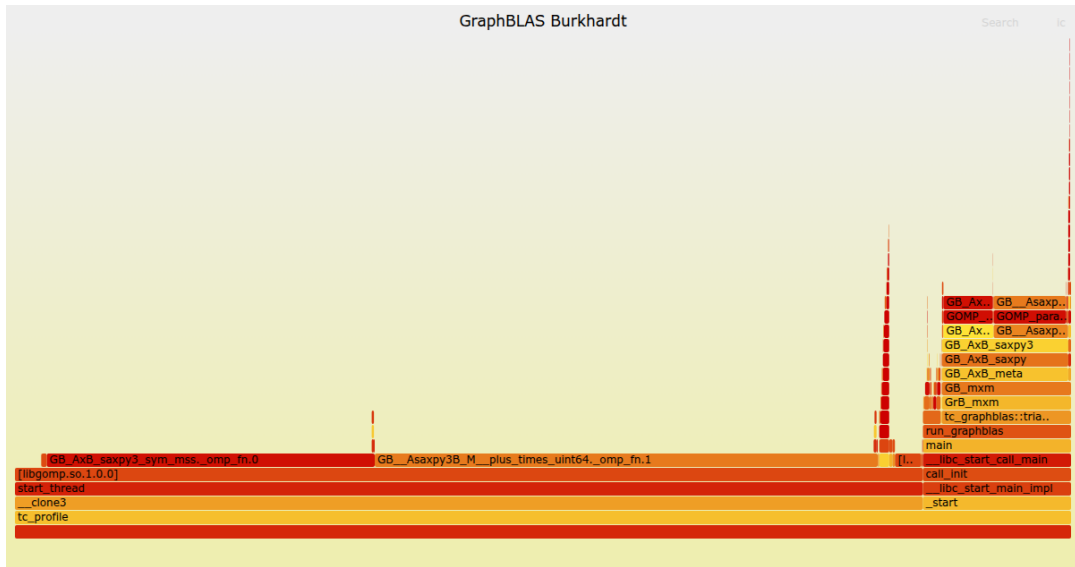
Эксперимент: Sandia — время выполнения



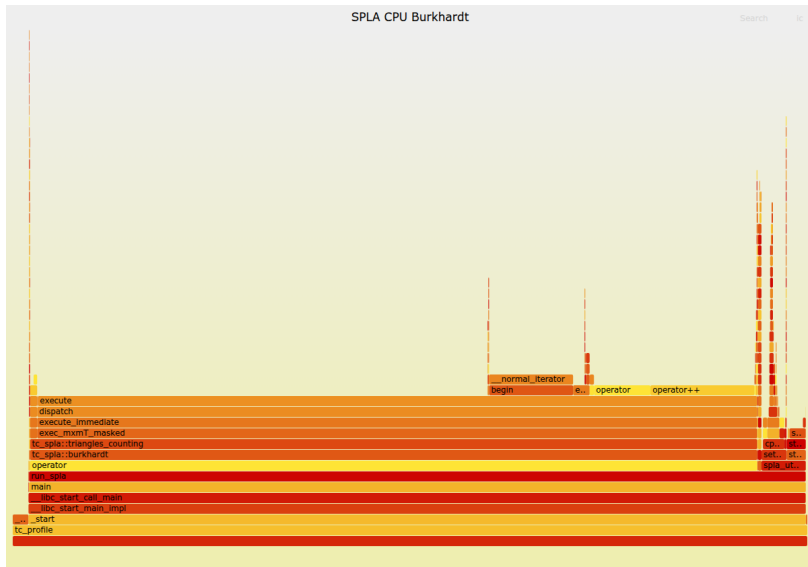
Эксперимент: Burkhardt & Sandia — время выполнения



Эксперимент: GraphBLAS Burkhardt FlameGraph



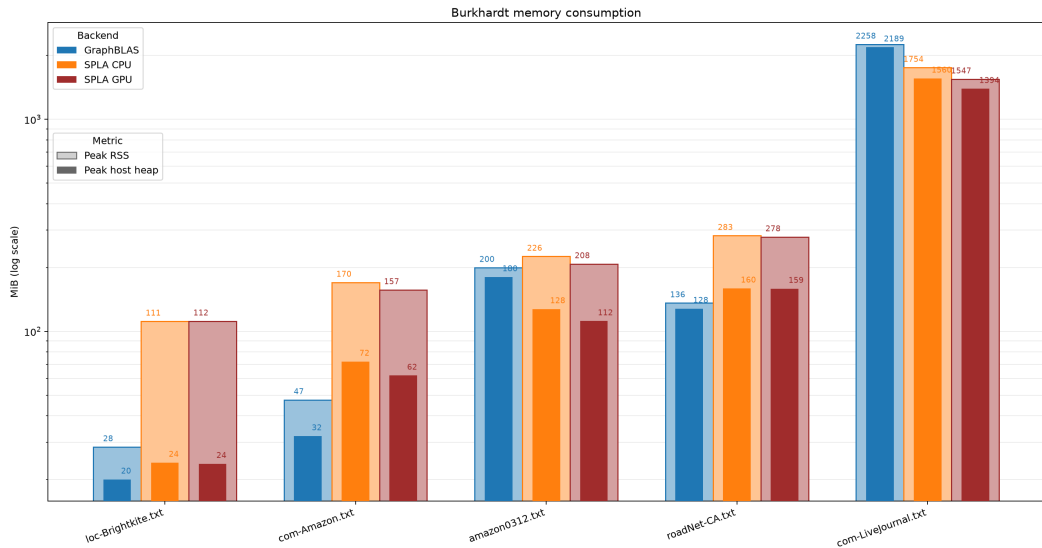
Эксперимент: SPLA Burkhardt FlameGraph



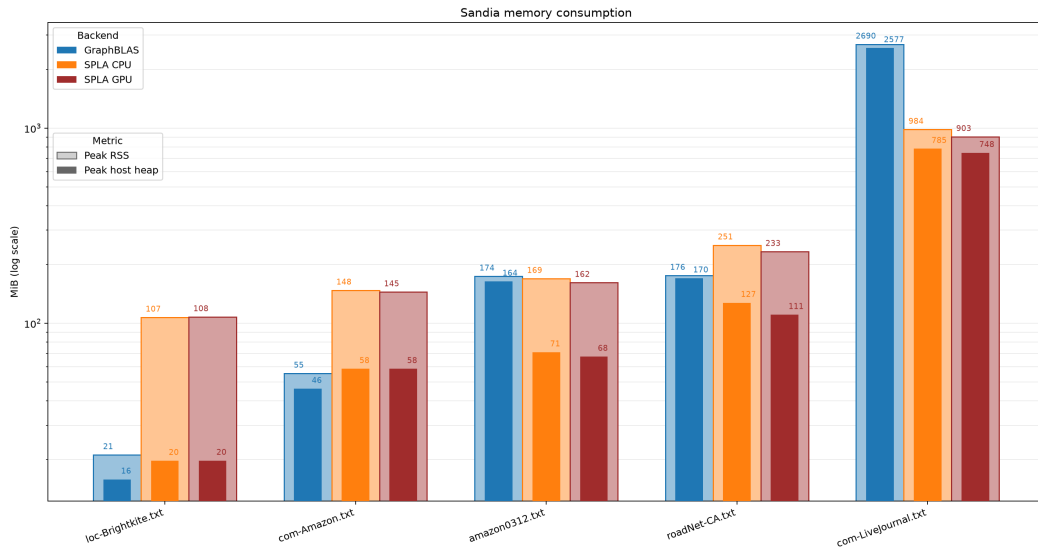
Эксперимент: время выполнения — результаты

- SPLA GPU показал минимальное время для обоих алгоритмов на всех графах
- GraphBLAS быстрее SPLA CPU благодаря параллельному специализированному ядру
- Sandia быстрее Burkhardt для каждого backend и датасета
- На крупных графах порядок реализаций сохраняется: SPLA GPU, GraphBLAS, SPLA CPU

Эксперимент: Burkhardt — потребление памяти



Эксперимент: Sandia — потребление памяти



Эксперимент: потребление памяти — результаты

- На малых графах GraphBLAS имеет наименьший базовый peak RSS
- Sandia сокращает потребление памяти на крупных графах